Project 3 - DSA 8020

Saransh Rakshak

Due: Thursday May 1, 2025

Contents

Project Description

The ERA-Interim is a global atmospheric reanalysis dataset. Reanalysis refers to the process of producing spatially and temporally gridded datasets through data assimilation, primarily for climate monitoring and analysis. In the R data file **NA_MaxT_ERA.RData**, you will find daily maximum temperature values from *1979* to *2017*, with a spatial resolution of approximately 80 km. The original global dataset has been subsetted to a region covering the contiguous United States and extending into Canada and Mexico.

In Problems 1 and 2, you are required to analyze an ERA-Interim average temperature time series at yearly and monthly temporal resolutions at a spatial location of your choice.

Problem 1. Analyzing Yearly Average Temperatures Time-Series

1.1. The following steps are needed to extract the annual mean temperatures at the specified location from the original dataset with daily temperatures at thousands of locations:

a. First, use the following R script to load the R data object:

```
load("NA_MaxT_ERA.RData")
NA_MaxT <- NA_MaxT - 273.15 # change from Kelvin to Celsius
nlon <- length(NA_lon); nlat <- length(NA_lat)
nmon <- 12; nyr <- 39 # numbers of month/year
```

b. Second, choose a location of interest (with the information of longitude and latitude) and identify the nearest ERA grid cell to the chosen location. Below you can find a script to get the correct ERA grid cell for Clemson:

```r
# Clemson Example
Clemson.lon.lat <- c(-82.8374, 34.6834) # (lon,lat)
dist2Clemson <- rdist.earth(
  matrix(Clemson.lon.lat, 1, 2),
  expand.grid(NA_lon, NA_lat),
  miles = F
  )
id <- which.min(dist2Clemson)
(lon_id <- id %% nlon)
```

```
## [1] 58
```

```r
(lat_id <- id %/% nlon + 1)
```

```
## [1] 15
```

```r
# Check
(rdist.earth(matrix(Clemson.lon.lat, 1, 2),
matrix(c(NA_lon[lon_id], NA_lat[lat_id]), 1, 2), miles = F)
== min(dist2Clemson))
```

```
##      [,1]
## [1,] TRUE
```

```r
Monterey.lon.lat <- c(-121.8979, 36.5973) # (lon,lat)
dist2Monterey <- rdist.earth(
  matrix(Monterey.lon.lat, 1, 2),
  expand.grid(NA_lon, NA_lat),
  miles = F
  )

id <- which.min(dist2Monterey)

(lon_id <- id %% nlon)
```

```
## [1] 5
```

```r
(lat_id <- id %/% nlon + 1)
```

```
## [1] 18
```

```r
(rdist.earth(
  matrix(Monterey.lon.lat, 1, 2),
  matrix( c(NA_lon[lon_id], NA_lat[lat_id]), 1, 2),
  miles = F
  )== min(dist2Monterey))
```

```
##      [,1]
## [1,] TRUE
```

c. Third, aggregate the daily data to the annual average time series. You could use the following script to achieve this:

```
time_series_daily <- NA_MaxT[lon_id, lat_id,]
print("Daily Time Series summary: ")
```

```
## [1] "Daily Time Series summary: "
```

```
print(summary(time_series_daily))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   4.173  13.178  14.816  14.864  16.539  24.845
```

```
time_series_yearly <- tapply(time_series_daily, list(yr), mean)
print("Yearly Time Series summary: ")
```

```
## [1] "Yearly Time Series summary: "
```

```
print(summary(time_series_yearly))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   14.06   14.44   14.74   14.86   15.26   16.18
```

1.2. Describe the main features of the annual time series data you extracted.

```
# tsfeatures for (feature autoextraction)
yearly_features <- tsfeatures(time_series_yearly)
print(yearly_features)
```

```
## # A tibble: 1 x 16
##   frequency nperiods seasonal_period trend    spike linearity curvature e_acf1
##       <dbl>    <dbl>           <dbl> <dbl>    <dbl>     <dbl>     <dbl>  <dbl>
## 1         1        0               1 0.434 0.000433    0.0857      2.28 0.0822
## # i 8 more variables: e_acf10 <dbl>, entropy <dbl>, x_acf1 <dbl>,
## #   x_acf10 <dbl>, diff1_acf1 <dbl>, diff1_acf10 <dbl>, diff2_acf1 <dbl>,
## #   diff2_acf10 <dbl>
```

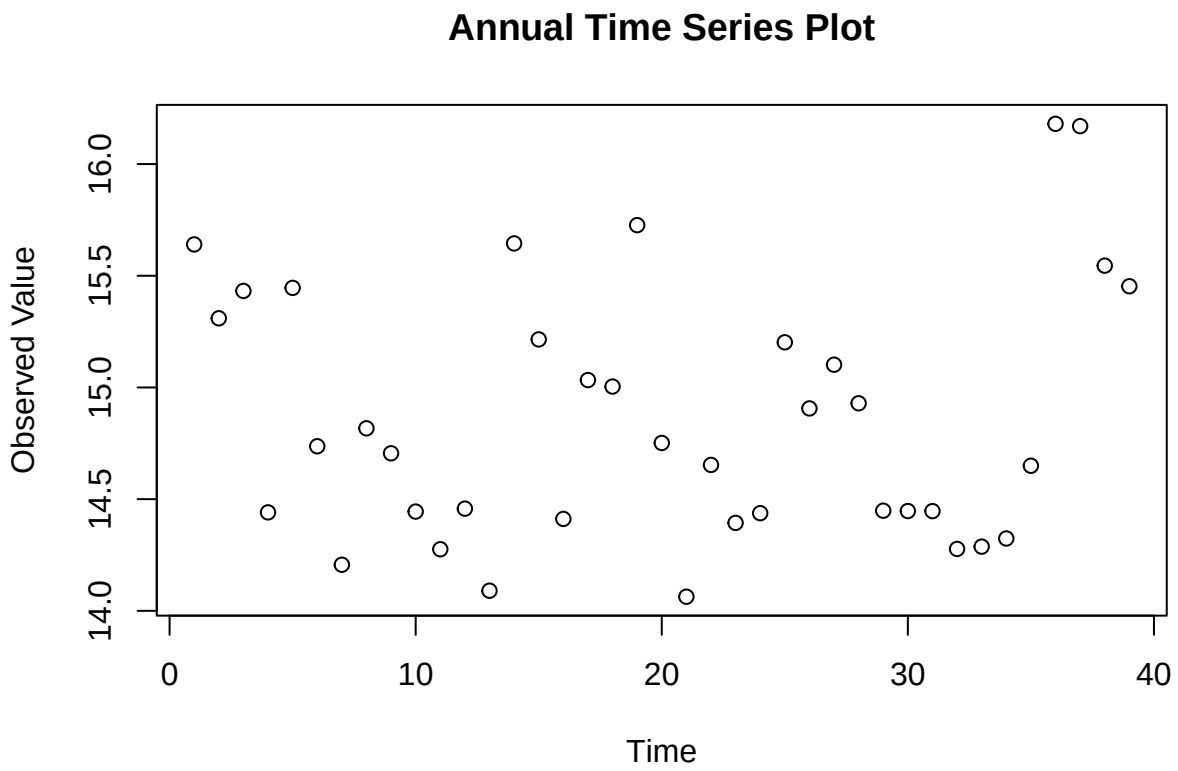
Our time series feature analysis using *tsfeatures* for the annual series reveals the following:

Feature	Value	Analysis
trend	0.4337	Moderately strong trend, indicates presence of long term pattern
spike	0.00042	Extremely low, indicates no significant anomalies or jumps in any given year
linearity	0.08572	Moderately low - some possibility model is linear

Feature	Value	Analysis
curvature	2.27771	High value strongly indicates non-linearity
entropy	0.9574	High value, indicates limited predictability and complex relations between the data
x_acf1	0.4117	Moderate indication of autocorrelation, any given years values show to depend on its previous year

1.3. Is it reasonable to assume that there is a linear trend? If so, estimate and remove the trend to get the detrended time series.

```
plot(time_series_yearly, main= "Annual Time Series Plot", ylab= "Observed Value", xlab= "Time")
```



Our graph does not reveal much significant information. However, based on our previous results from *tsfeatures*, it is still somewhat reasonable to expect the possibility of a linear trend:

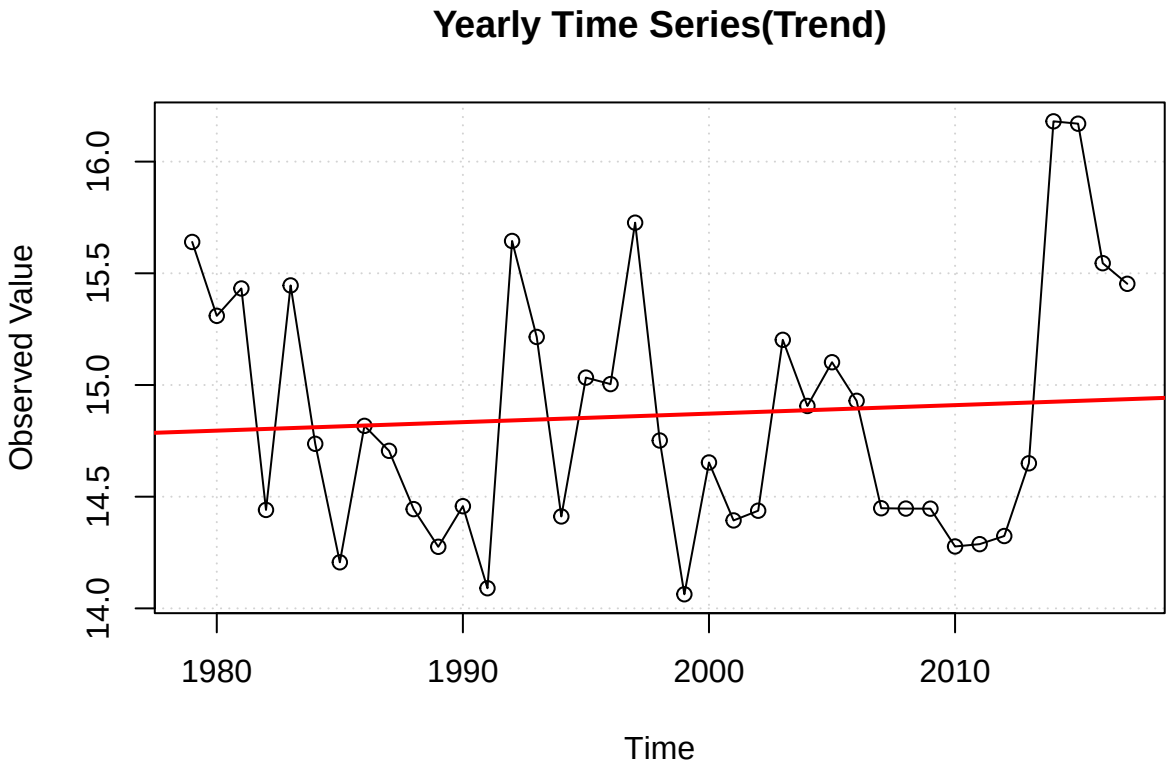
1. trend: value 0.43 shows significant enough trend in data to expect some prediction.
2. linearity: while this is not 0, it is still low enough to keep linearity as a reasonable possibility.
3. curvature: even though value is high (indicating a non-perfect linear fit), it is still significant enough to keep linearity a potential assumption, especially early on in the analysis.

Since a linear trend is not out of question for our time series, we will now estimate its value.

```r
yrs <- unique(yr)

# fitting linear model with trend
lm_annual <- lm(time_series_yearly ~ yrs)
summary(lm_annual)
```

```
##
## Call:
## lm(formula = time_series_yearly ~ yrs)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -0.8047 -0.4528 -0.1126  0.4420  1.2551
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  7.254522  16.272999   0.446    0.658
## yrs          0.003809   0.008145   0.468    0.643
##
## Residual standard error: 0.5724 on 37 degrees of freedom
## Multiple R-squared:  0.005876,   Adjusted R-squared:  -0.02099
## F-statistic: 0.2187 on 1 and 37 DF,  p-value: 0.6428
```

```r
plot(
  x= yrs,
  y= time_series_yearly,
  type= "o", # measured points w/ time continuity
  main= "Yearly Time Series(Trend)",
  ylab= "Observed Value",
  xlab= "Time",
  panel.first= grid()
  )
abline(lm_annual, col= "red", lwd= 2)
```

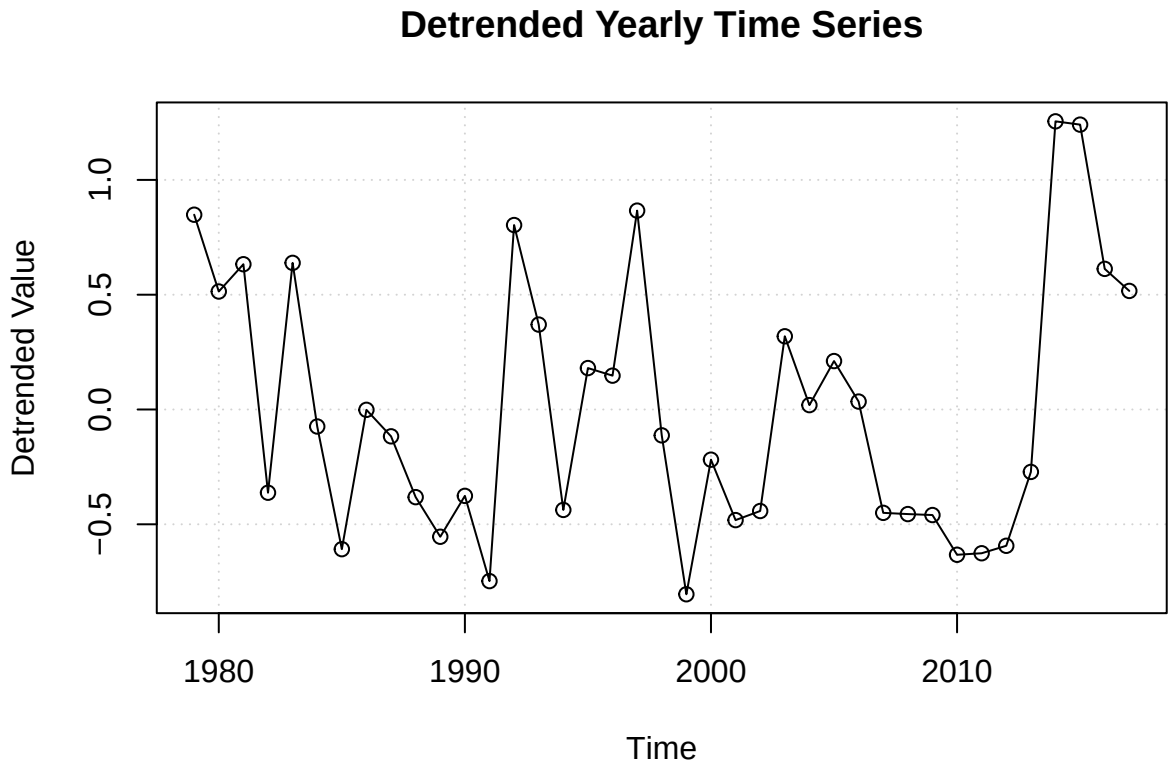


Our linear model fit reveals a p-value of 0.64, showing that a linear trend is statistically insignificant - leading us to fail to reject the null hypothesis and conclude there is no strong proof of linear behavior in the data. This is further confirmed by our Yearly Time Series(Trend) plot, which shows the data fluctuating significantly around our linear trend line (red) as well as many clearly non-linear patterns. It may be possible that the data could have a different measurable trend such as cyclical behavior, as is not unusual with environmental data.

1.4. Make acf and pacf plots for the detrended time series to identify possible ARMA models. That is, determine the possible orders p for the AR component and the orders q for the MA component.

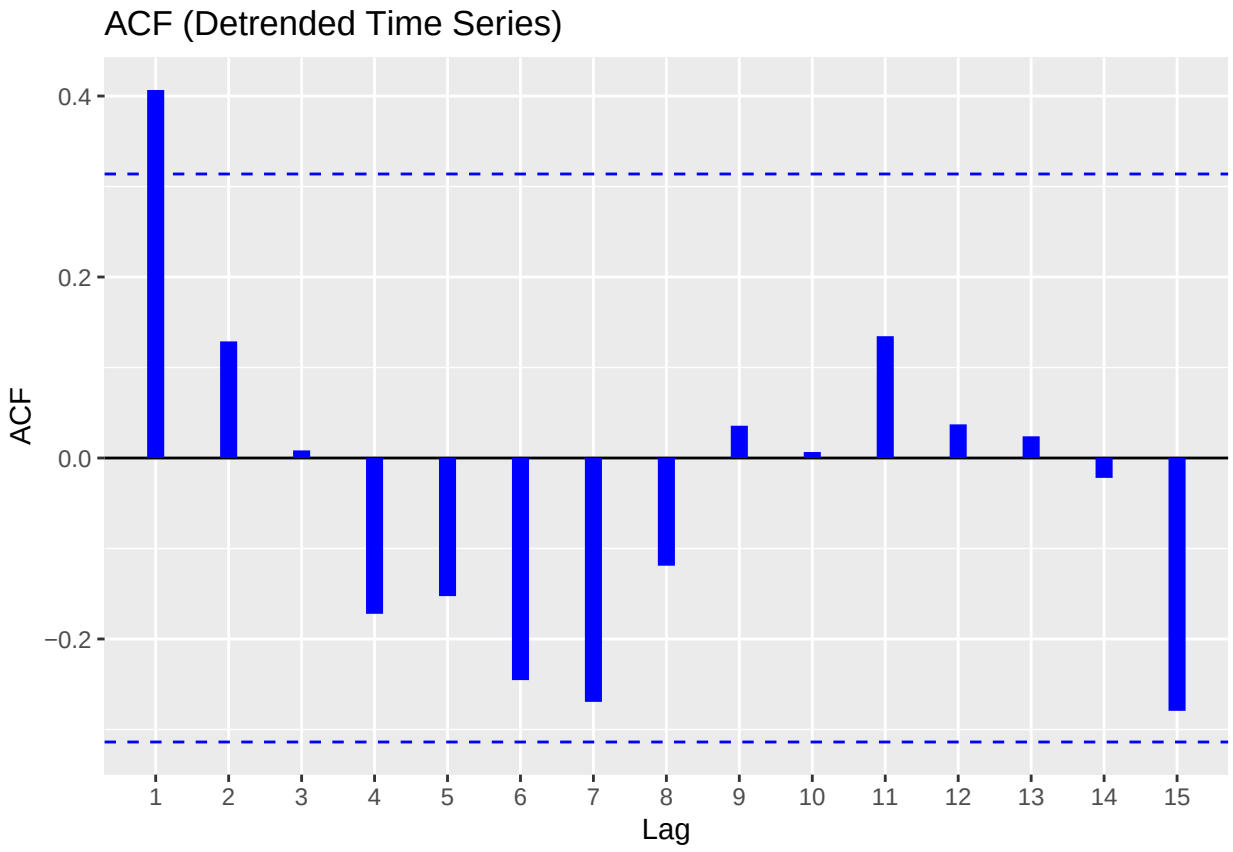
Although our linear trend model in part **1.3** was not statistically significant, we will proceed to detrend the time series data to obtain residual series which reflect deviation from potential linear trend. Our detrended model will provide useful for further decomposition or static testing. We will detrend our data by subtracting the fitted linear trend from observed values.

```
# detrended time series
detrended_ts <- residuals(lm_annual)
plot(
  yrs,
  detrended_ts,
  type= "o",
  main= "Detrended Yearly Time Series",
  ylab= "Detrended Value",
  xlab= "Time",
  panel.first= grid()
)
```



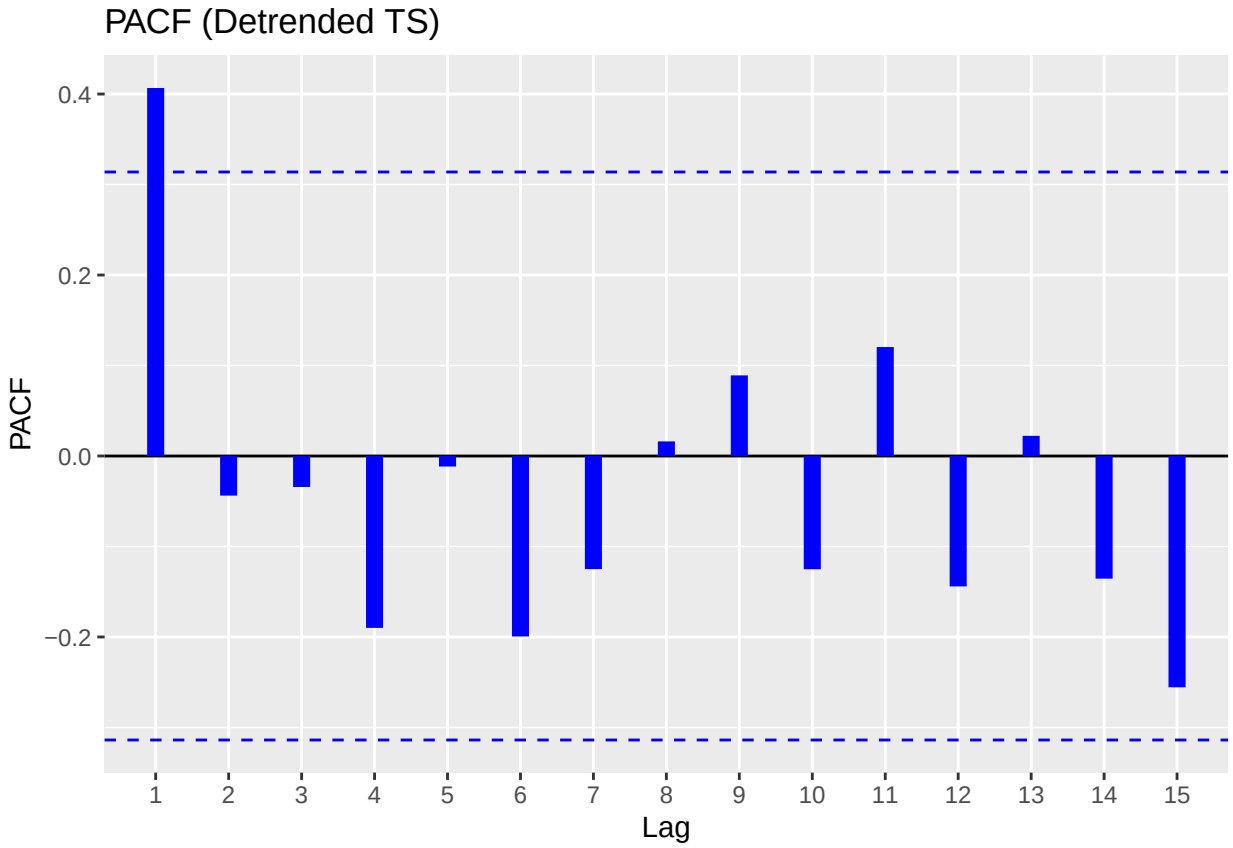

```
# ACF plot
ggAcf(detrended_ts,
      main = "ACF (Detrended Time Series)",
      size= 3, col= "blue")
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown
## parameters: ‘main‘
```




```
# PACF plot
ggPacf(detrended_ts,
       main = "PACF (Detrended TS)",
       size= 3, col= "blue")
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown
## parameters: ‘main‘
```



In determining a suitable ARMA model for our detrended time series, we examined the Auto-correlation and Partial auto-correlation function plot generated using residuals of the linear trend model.

ACF Observations:

- Shows significant spike at lag=1, which is followed by values which quickly taper off and fall within 95% confidence interval bound.
- This cutoff after lag=1 is native to a Moving Average process, with order q=1 $\rightarrow$ suggesting MA(1) as potential.

PACF Observations:

- Also shows significant spike at lag=1, however subsequent lags do not show substantial values.
- This quality is characteristic of Auto-regressive processes of order p=1 $\rightarrow$ suggests a potential AR(1) component.

Model Implications:

Since both ACF and PACF show prominent spike at lag=1 and no significant subsequent autocorrelation, we can establish a model as follows:

ARMA(1,1): Both MA(1) and AR(1) components are relavent.

However, we will also establish simpler models, such as *AR(1)* and *MA(1)* for comparing using model selection criteria (AIC/BIC).

1.5. Fit the possible ARMA models you identified and perform model selection to determine the 'best' model.

To identify our optimal ARMA model for our detrended time series, we will fit our three candidate models we found from ACF/PACF plot analysis.

- AR(1): Autoregressive model of order 1.

- MA(1): Moving average model of order 1.

- ARMA(1,1): Combined model autoregressive + moving average.

We will use the *Arima()* function from the **forecast** library to fit each ARMA model and determine optimal choice by comparing AIC/BIC values.

```
# fit AR(1), MA(1), ARMA(1,1)
model_ar1 <- Arima(detrended_ts, order= c(1,0,0))
model_ma1 <- Arima(detrended_ts, order= c(0,0,1))
model_arma11 <- Arima(detrended_ts, order= c(1,0,1))

# summary(model_arma11)

# AIC vs BIC to pick best model
model_comparison <- data.frame(
  Model = c("AR(1)", "MA(1)", "ARMA(1,1)"),
  AIC = c(AIC(model_ar1), AIC(model_ma1), AIC(model_arma11)),
  BIC = c(BIC(model_ar1), BIC(model_ma1), BIC(model_arma11))
)

print(model_comparison)
```

```
##       Model      AIC      BIC
## 1     AR(1) 63.54890 68.53959
## 2     MA(1) 63.97596 68.96665
## 3 ARMA(1,1) 65.51581 72.17006
```

Results:

- The AR(1) model has lowest AIC/BIC values by a small margin, indicating it offers the best trade off between good fit and model complexity.
- MA(1) very close in performance.
- ARMA(1,1) clearly not an optimal choice.

Based on model selection criteria, we will select AR(1) model as our choice for modeling this time series. This implies a linear relationship with last year's anomaly alone best explains the current year's temperature anomaly.

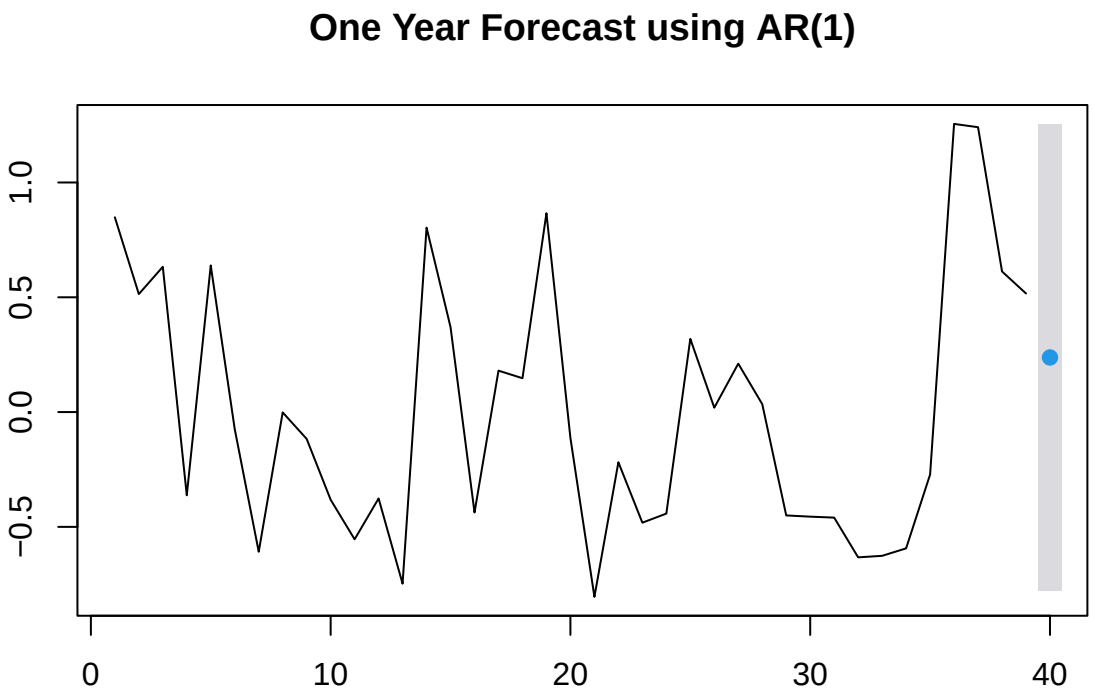
1.6. Perform a one-year-ahead forecast. Calculate the prediction and provide a 95% Prediction Interval.

Using the AR(1) model, we will perform a one year forecast to predict the next value in our detrended anomaly series. We will use the *forecast()* function with 'h=1' to compute forecast for one time step ahead, as well as specify a 95% Prediction Interval to quantify uncertainty around forecast. As our AR(1) model was built using detrended time series data, the following forecast will be for anomalies only.

```
forecast_ar1 <- forecast(model_ar1, h= 1, level= 95)
print(forecast_ar1)
```

```
##    Point Forecast      Lo 95    Hi 95
## 40      0.2378151 -0.7777692 1.253399
```

```
plot(forecast_ar1, main= "One Year Forecast using AR(1)")
```



Forecasting Results:

We can predict a temperature anomaly for next year to be approximately 0.2378 °C. Our 95% Prediction Interval ranges from −0.778 °C to 1.253 °C, meaning we are 95% confident values will fall in this range. The positive point forecast alludes a mild increase in temperature anomaly which continues slow upwards patter of recent years. The wide nature of the prediction interval can be reflective of external factors that our model could not account for, and also highlights uncertainty in long term forecasting from lower order AR models, which prove to be more useful for short term predictions.

Problem 2. Analyzing Monthly Average Temperature Time Series

Repeat the analysis for Problem 1 but for the monthly average time series (which can be extracted using the script below). An important component of this problem is the analysis of seasonal patterns when dealing with sub-annual data. Some techniques from Week 14 should be employed to complete this problem.

2.2. Describe the main features of the monthly time series data you extracted.

```
time_series_monthly_matrix <- tapply(time_series_daily, list(mon, yr), mean)
time_series_monthly <- ts(as.vector(time_series_monthly_matrix),
                          start = c(min(yr),
                                    min(mon)),
                          frequency = 12)
monthly_features <- tsfeatures(time_series_monthly)

print("Monthly Time Series summary: ")
```

```
## [1] "Monthly Time Series summary: "
```

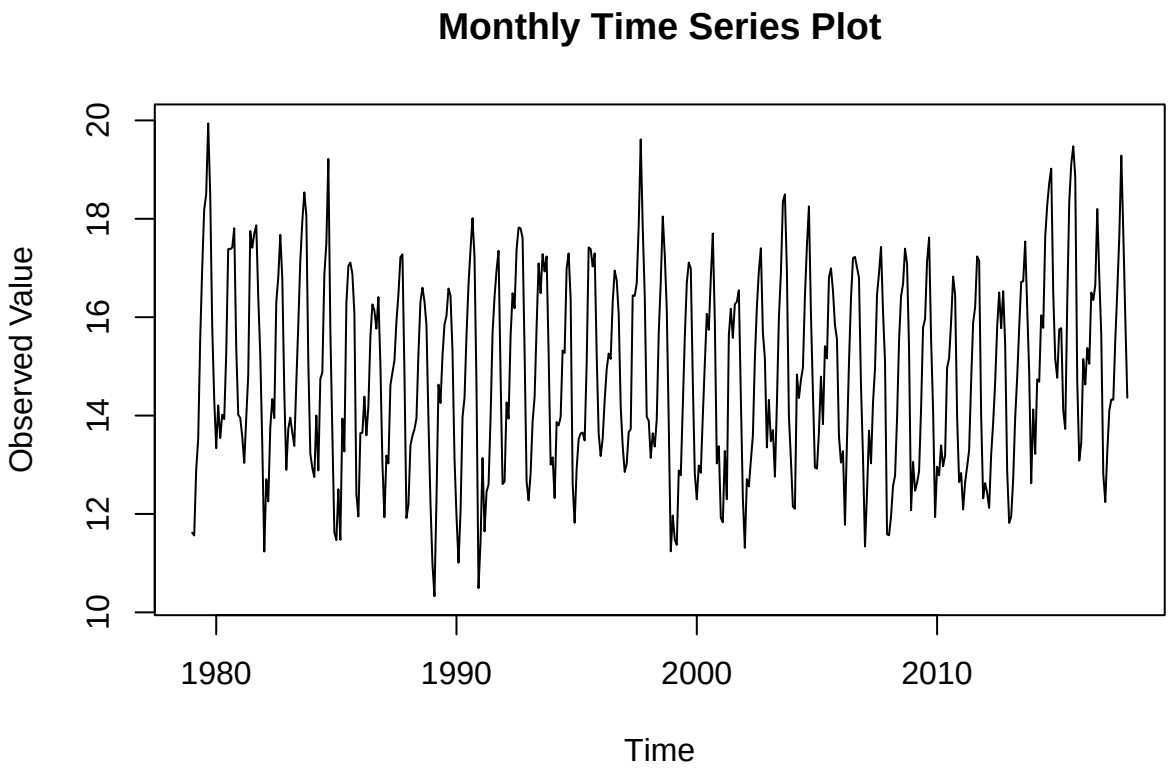
```
print(summary(time_series_monthly))
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##   10.33   13.23   14.77   14.86   16.45   19.94
```

```
print(monthly_features)
```

```
## # A tibble: 1 x 20
##   frequency nperiods seasonal_period trend      spike linearity curvature e_acf1
##       <dbl>    <dbl>           <dbl> <dbl>      <dbl>     <dbl>     <dbl>  <dbl>
## 1        12        1              12 0.506    8.56e-8     0.555      2.55  0.208
## # i 12 more variables: e_acf10 <dbl>, seasonal_strength <dbl>, peak <dbl>,
## #   trough <dbl>, entropy <dbl>, x_acf1 <dbl>, x_acf10 <dbl>, diff1_acf1 <dbl>,
## #   diff1_acf10 <dbl>, diff2_acf1 <dbl>, diff2_acf10 <dbl>, seas_acf1 <dbl>
```

```
plot(time_series_monthly, main = "Monthly Time Series Plot", ylab = "Observed Value", xlab = "Time")
```



Our time series feature analysis using *tsfeatures* for the monthly series reveals the following:

Feature	Value	Analysis
frequency	12	Shows strong monthly and seasonal patterns - as expected in climate data.
season	12	Shows strong monthly and seasonal patterns - as expected in climate data.
trend	0.50617	Moderate trend can be detected.
spike	8.56e-08	Extremely low, shows no outliers in time series data.
linearity	0.555405	Points to nonlinearity, meaning other patterns such as seasonal or cyclical could be at play.
curvature	2.554952	Points to nonlinearity, meaning other patterns such as seasonal or cyclical could be at play.
entropy	0.497834	Shows high levels of randomness/unpredictability.

The Monthly TS Plot clearly shows patterns of seasonality - attributed to the periodic peaks and drops

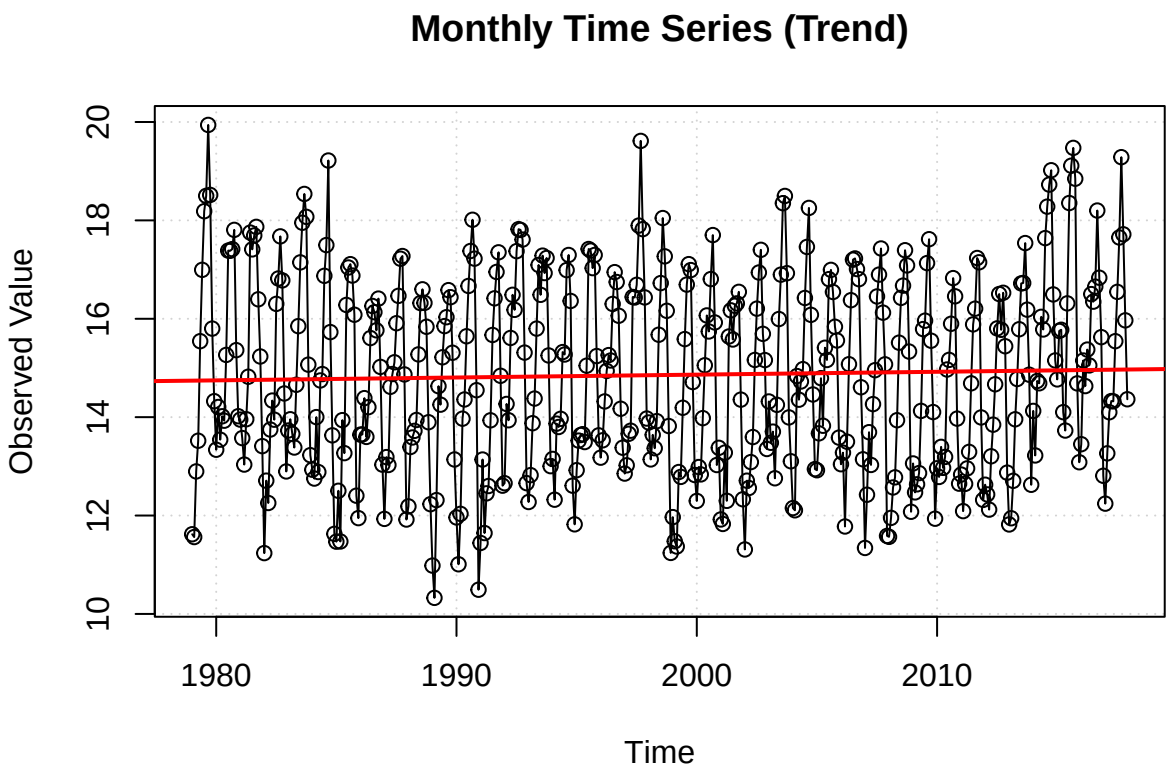
2.3. Is it reasonable to assume that there is a linear trend? If so, estimate and remove the trend to get the detrended time series.

```
time_index <- time(time_series_monthly)
lm_monthly <- lm(time_series_monthly ~ time_index)

summary(lm_monthly)
```

```
##
## Call:
```

```
## lm(formula = time_series_monthly ~ time_index)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -4.4740 -1.6431 -0.1058  1.5824  5.1941
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)  3.239169  16.110097   0.201    0.841
## time_index   0.005813   0.008061   0.721    0.471
##
## Residual standard error: 1.963 on 466 degrees of freedom
## Multiple R-squared:  0.001114,   Adjusted R-squared:  -0.001029
## F-statistic: 0.5199 on 1 and 466 DF,  p-value: 0.4712
```

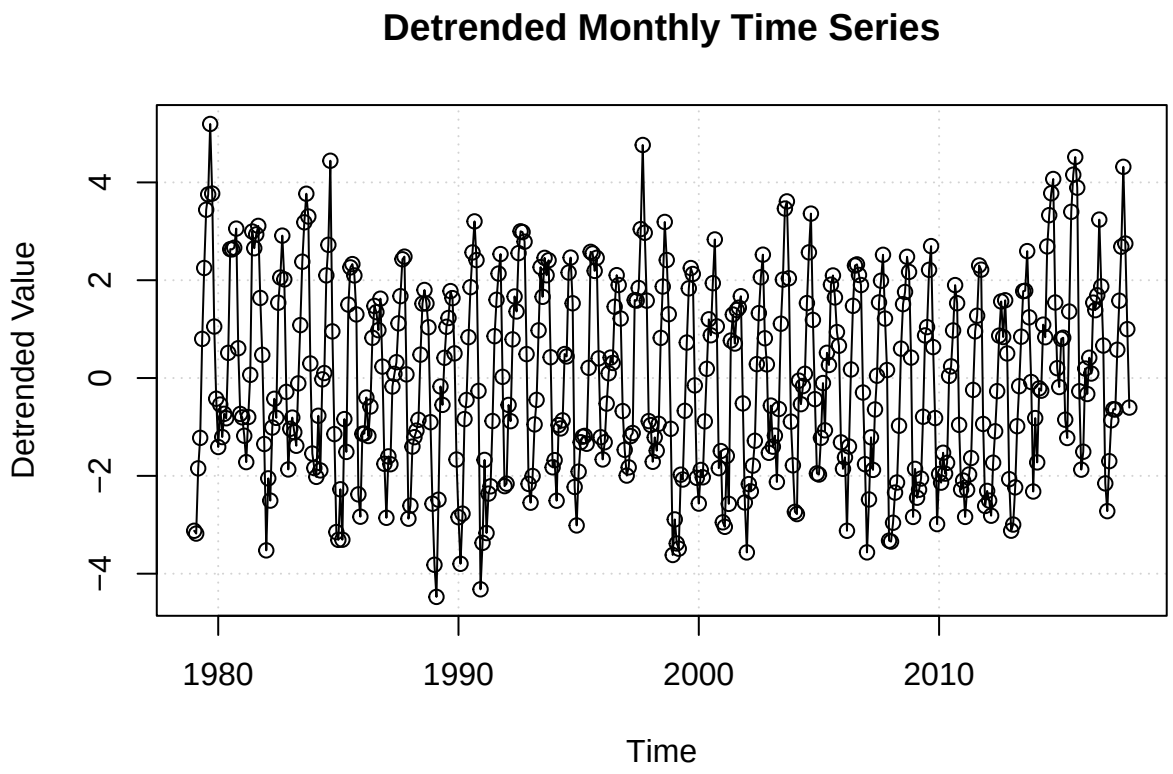
```r
plot(
  x= time_index,
  y= time_series_monthly,
  type= "o", # measured points w/ time continuity
  main= "Monthly Time Series (Trend)",
  ylab= "Observed Value",
  xlab= "Time",
  panel.first= grid()
  )
abline(lm_monthly, col = "red", lwd = 2)
```



There is no strong evidence that the series shows any type of linear trend as it's p-value is very high (0.4712). This is reflected in the plot, where points rarely follow a linear pattern. Thus we will not go through the process of detrending the time series.

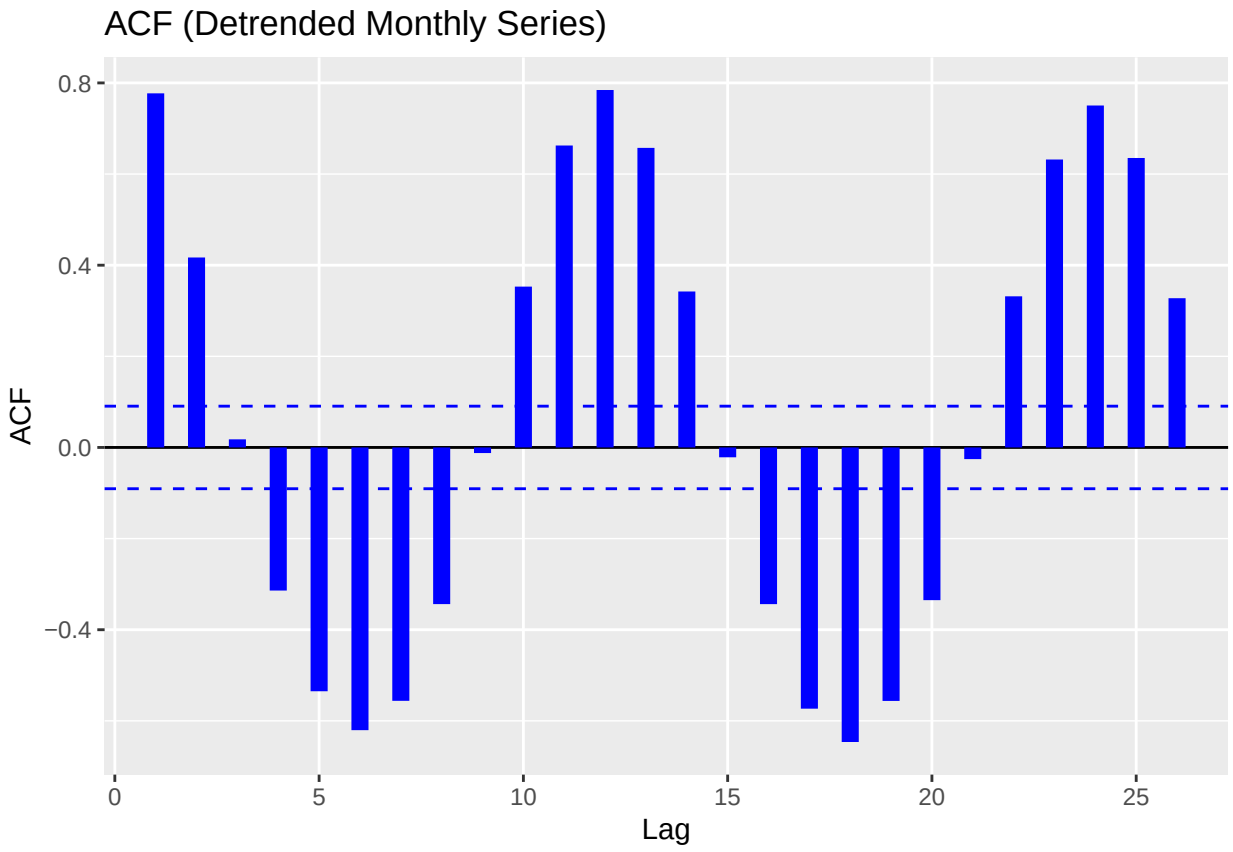
2.4. Make acf and pacf plots for the detrended time series to identify possible ARMA models. That is, determine the possible orders p for the AR component and the orders q for the MA component.

```
detrended_monthly_ts <- residuals(lm_monthly)
plot(
  time_index,
  detrended_monthly_ts,
  type= "o",
  main= "Detrended Monthly Time Series",
  ylab= "Detrended Value",
  xlab= "Time",
  panel.first= grid()
)
```



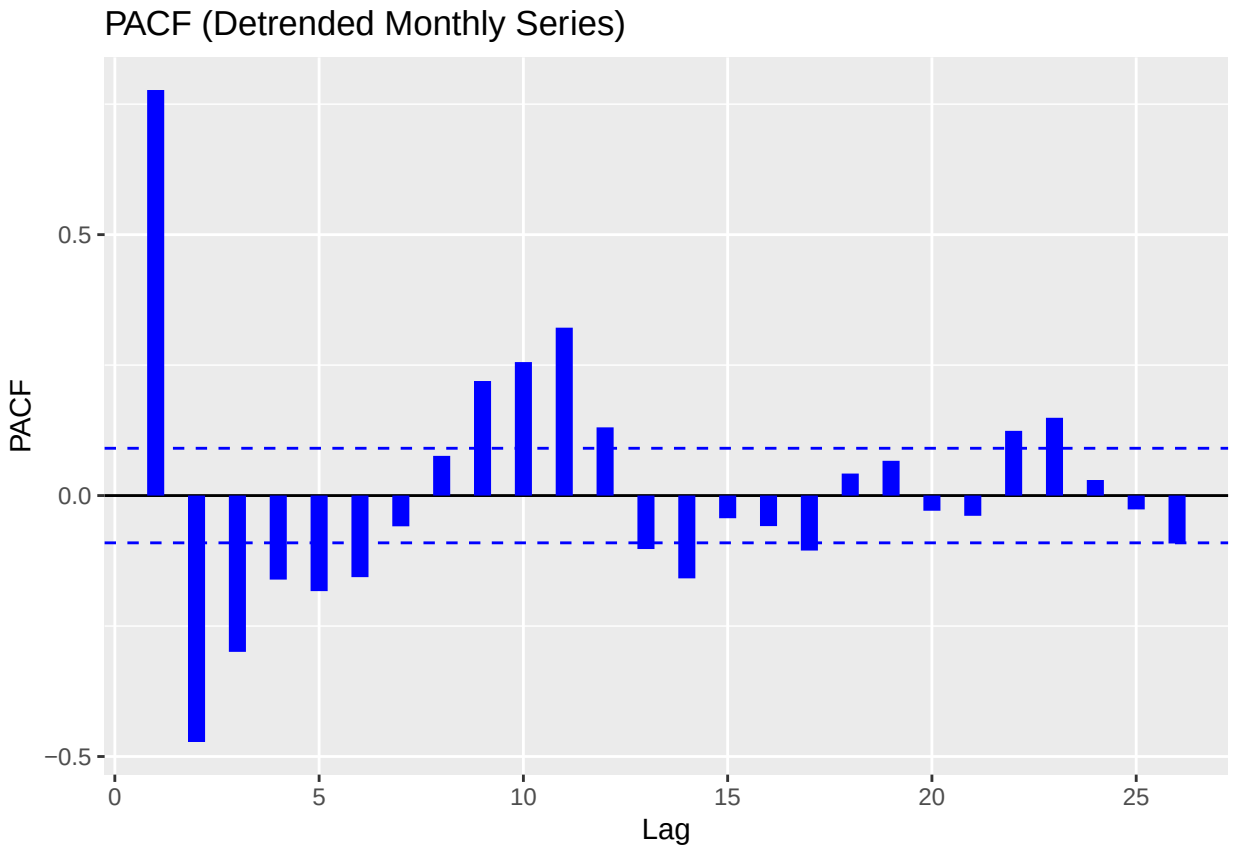

```
# ACF plot
ggAcf(detrended_monthly_ts, main = "ACF (Detrended Monthly Series)", size = 3, col = "blue")
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown
## parameters: `main`
```




```
# PACF plot
ggPacf(detrended_monthly_ts, main = "PACF (Detrended Monthly Series)", size = 3, col = "blue")
```

```
## Warning in ggplot2::geom_segment(lineend = "butt", ...): Ignoring unknown
## parameters: `main`
```



ACF Plot:

```
- Strong spike at lag 12, indicating seasonality.
- All other lags have relatively moderate change between lags.
- Good candidates would be MA(1), MA(2)
```

PACF Plot:

```
- Strong spike at lag 1: AR(1) candidate.
```

Our plots reveal the best candidate models for fitting are:

```
- AR(1)
- MA(1), MA(2)
- ARMA(1,1), ARMA(1,2)
```

2.5. Fit the possible ARMA models you identified and perform model selection to determine the 'best' model.

```r
# fit AR(1), MA(1/2), ARMA(1,1/2)
month_model_ar1 <- Arima(detrended_monthly_ts, order=c(1,0,0))
month_model_ma1 <- Arima(detrended_monthly_ts, order=c(0,0,1))
month_model_ma2 <- Arima(detrended_monthly_ts, order=c(0,0,2))
```

```r
month_model_arma11 <- Arima(detrended_monthly_ts, order=c(1,0,1))
month_model_arma12 <- Arima(detrended_monthly_ts, order=c(1,0,2))


summary(month_model_arma12)
```

```
## Series: detrended_monthly_ts
## ARIMA(1,0,2) with non-zero mean
##
## Coefficients:
##          ar1     ma1     ma2     mean
##       0.5439  0.5039  0.3157  -0.0165
## s.e.  0.0535  0.0571  0.0479   0.2022
##
## sigma^2 = 1.222:  log likelihood = -709.62
## AIC=1429.25   AICc=1429.38   BIC=1449.99
##
## Training set error measures:
##                       ME     RMSE       MAE       MPE     MAPE      MASE
## Training set 0.004355508 1.100664 0.8934256 -7.330998 143.3993 0.8667917
##                     ACF1
## Training set 0.03409115
```

```r
# AIC vs BIC to pick best model
model_comparison <- data.frame(
  Model = c("AR(1)", "MA(1)", "MA(2)", "ARMA(1,1)", "ARMA(1,2)"),
  AIC = c(AIC(month_model_ar1),
          AIC(month_model_ma1),
          AIC(month_model_ma2),
          AIC(month_model_arma11),
          AIC(month_model_arma12)
          ),
  BIC = c(BIC(month_model_ar1),
          BIC(month_model_ma1),
          BIC(month_model_ma2),
          BIC(month_model_arma11),
          BIC(month_model_arma12))
)

print(model_comparison)
```

```
##       Model      AIC      BIC
## 1     AR(1) 1527.025 1539.471
## 2     MA(1) 1631.041 1643.486
## 3     MA(2) 1484.672 1501.266
## 4 ARMA(1,1) 1464.162 1480.756
## 5 ARMA(1,2) 1429.247 1449.989
```

AMRA(1,2) achieved the lowest AIC/BIC criterion values, outperforming our AR(1) MA(1/2) and ARMA(1,1) models. The reason for its success is likely its increased flexibility useful for seasonal/cyclical data. The specific choice in ARMA(1,2) indicates that the monthly time series shows short term autocorrelation (achieved by AR(1)) as well as requires corrections for noise across two periods (MA(2)) showing slightly longer dependence in the series further than short term immediate months.

The significantly higher AIC/BIC values compared to the yearly series once again confirms increased complexity in the monthly series. This is expected as monthly data naturally contains more variation/short-term rapid changes which can't be captured by simpler models like AR(1), MA(1), MA(2), or even ARMA(1,1).

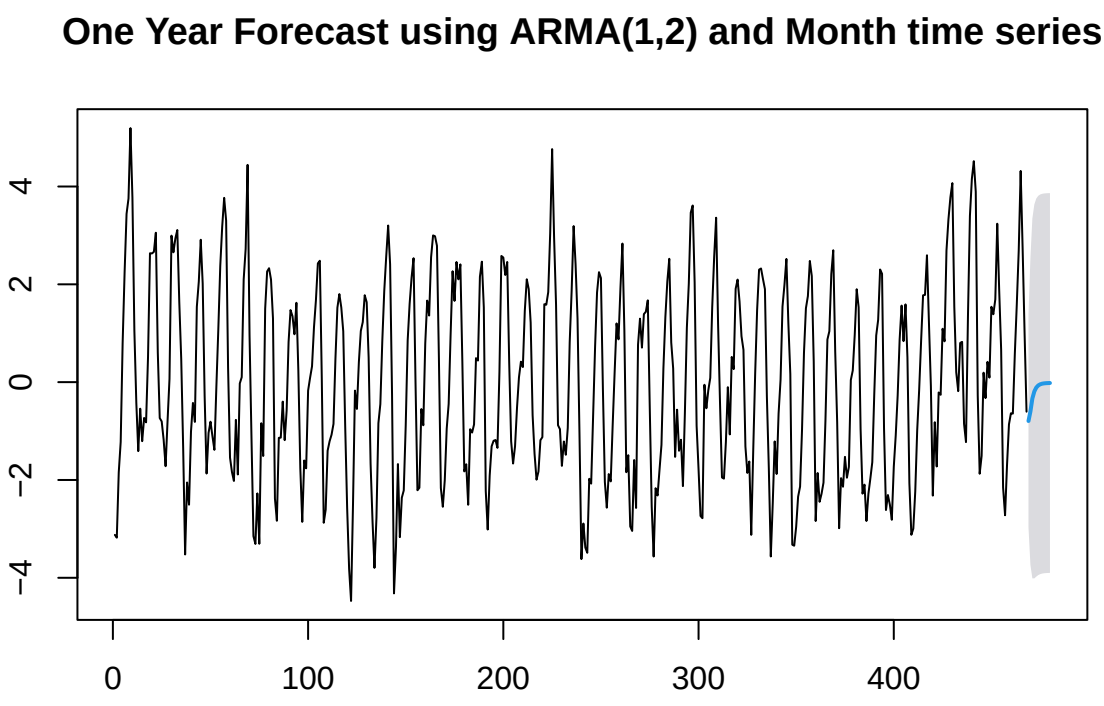
ARMA(1,2) is the selected model moving forward.

2.6. Perform a one-year-ahead forecast. Calculate the prediction and provide a 95% Prediction Interval.

```
month_forecast_arma12 <- forecast(month_model_arma12, h = 12, level = 95) # h=12 for full year
print(month_forecast_arma12)
```

```
##     Point Forecast     Lo 95    Hi 95
## 469    -0.79460459 -2.961146 1.371936
## 470    -0.59927753 -3.737209 2.538654
## 471    -0.33346298 -4.011362 3.344436
## 472    -0.18889691 -4.011922 3.634128
## 473    -0.11027310 -3.975180 3.754634
## 474    -0.06751270 -3.944721 3.809696
## 475    -0.04425701 -3.925097 3.836583
## 476    -0.03160915 -3.913522 3.850304
## 477    -0.02473047 -3.906961 3.857500
## 478    -0.02098943 -3.903314 3.861335
## 479    -0.01895483 -3.901307 3.863397
## 480    -0.01784829 -3.900209 3.864512
```

```
plot(month_forecast_arma12, main= "One Year Forecast using ARMA(1,2) and Month time series")
```



Problem 3. Spatial Interpolation Using Gaussian process model.

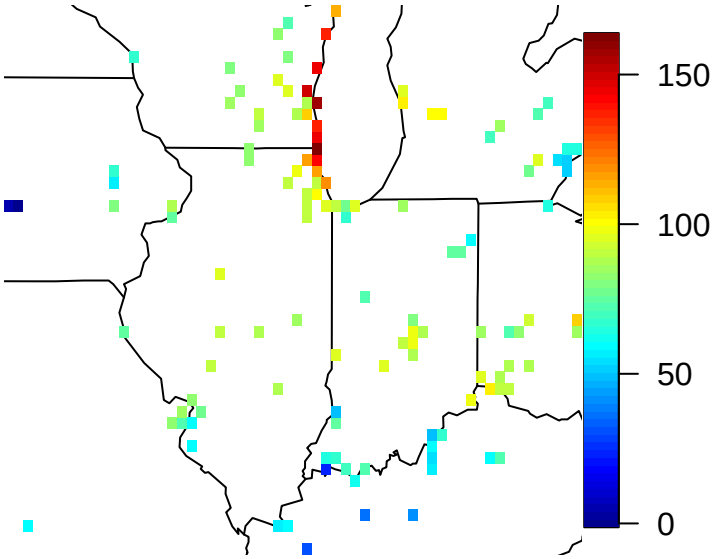
In this problem we will use the fields package to conduct spatial interpolation using Gaussian process model. We use ozone2 dataset in the fields package to interpolate average ozone levels.

3.1. Use the following script to load the data:

```
data(ozone2)
loc <- ozone2$lon.lat
rg <- apply(loc, 2, range)
y <- ozone2$y[16,]
good <- !is.na(y)
```

3.2. Visualize the spatial data using the script below and describe your findings:

```
library(maps)
map("state", xlim = rg[, 1], ylim = rg[, 2])
quilt.plot(loc[good,], y[good], nx = 60, ny = 48, add = T)
```

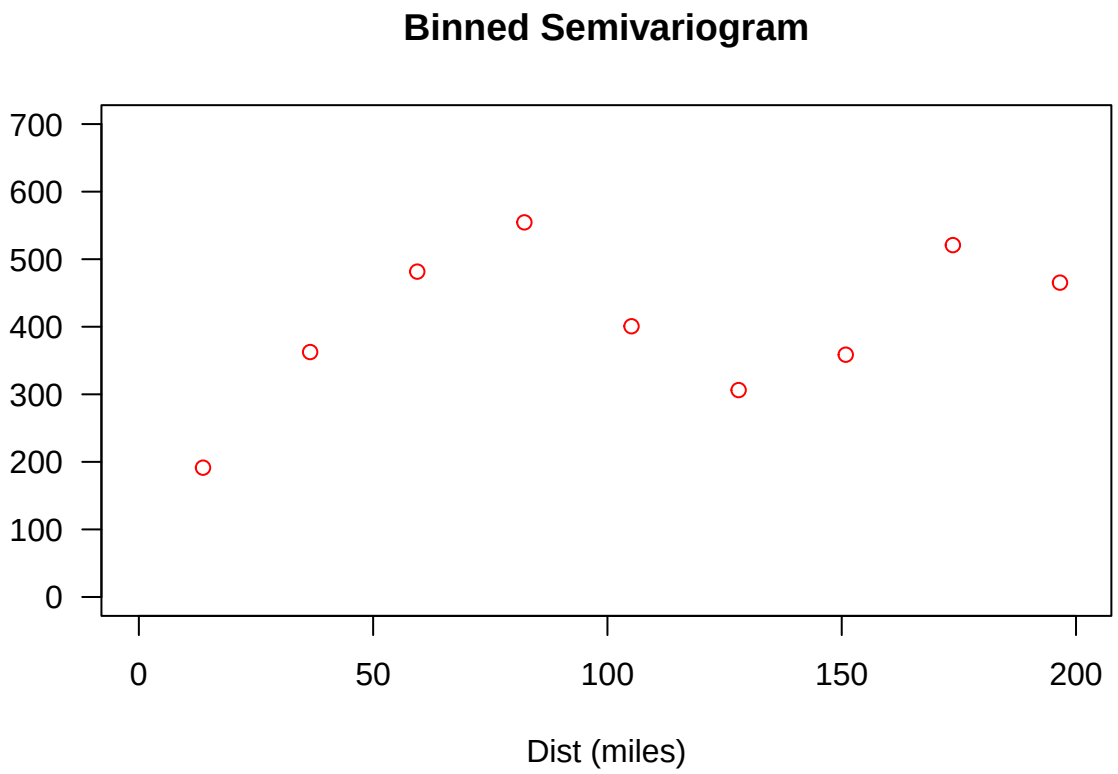


Findings:

The spatial plot produced using `quilt.plot()` overlays observed ozone measurements on a map of the US. The color gradient reveals a clear spatial structure in ozone levels across locations. Higher concentrations appear in certain clustered areas while other regions show consistently lower values. This indicates spatial heterogeneity in the ozone data, which justifies the need for spatial modeling techniques like Gaussian process regression. Additionally, there appears to be a smooth transition in values across space, suggesting underlying spatial correlation. This is a key assumption that will be further validated in the variogram analysis.

3.3. Make a variogram by assuming a linear spatial trend. You may use the following script to plot the variogram. Describe qualitatively what you learned about the spatial correlation from this variogram.

```
# remove linear spatial trend
lm <- lm(y[good] ~ loc[good,])
d <- rdist.earth(loc[good,]); maxd <- max(d)
vgram <- vgram(loc = loc[good,], y = lm$residuals, N = 30, lon.lat = TRUE)
plot(vgram$stats["mean",] ~ vgram$centers,
     main = "Binned Semivariogram",
     las = 1,
     ylab = "",
     xlab = "Dist (miles)",
     col = "red",
     xlim = c(0, 0.3 * maxd),
     ylim = c(0, 700)
     )
```



Findings:

The binned semi-variogram shows a clear upward trend with increasing distance, which is characteristic of spatial correlation: observations that are closer together tend to be more similar than those farther apart. Initially, the semi-variance increases rapidly, indicating strong spatial correlation at small distances. However, the curve begins to level off around a distance of 150–200 miles, suggesting a potential range beyond which spatial correlation diminishes significantly (i.e., spatial dependence weakens).

By first removing a linear spatial trend before computing the variogram, we isolate and visualize the residual spatial correlation that is not explained by the trend. This makes the variogram a better diagnostic for the spatial error component, allowing for a clearer identification of spatial correlation structure in the data.

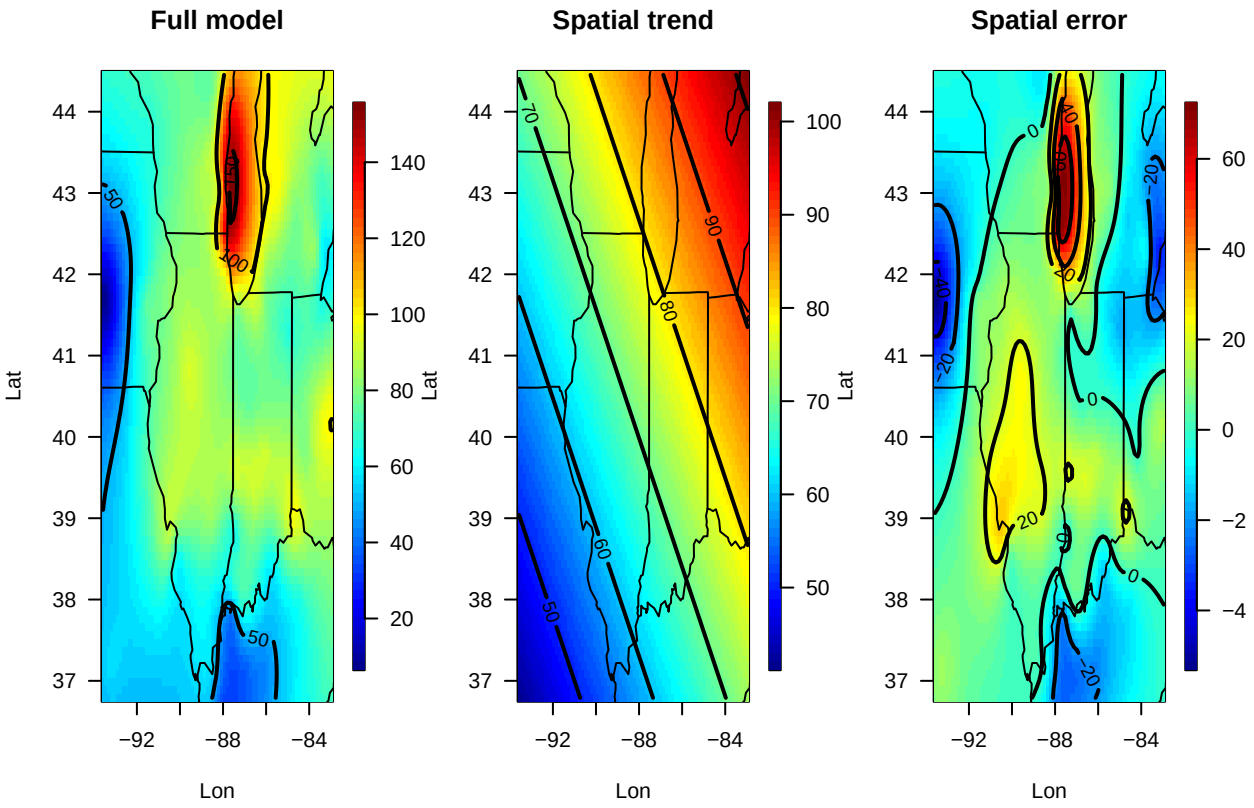
Without removing the trend, the variogram could misleadingly reflect large-scale drift instead of localized correlation.

3.4. Use the following script to fit a Gaussian process to the data and to decompose the prediction into a spatial trend part and spatially correlated error part. Explain what mean function and covariance function are being used here.

```
fit <- spatialProcess(loc[good,], y[good])

out.full <- predictSurface(fit, extrap = T)
out.poly <- predictSurface(fit, just.fixed = T, extrap = T)

out.spatial <- out.full
out.spatial$z <- out.full$z - out.poly$z

set.panel(1, 3)
```

```
## plot window will lay out plots in a 1 by 3 matrix
```

```
surface(out.full, las = 1, xlab = "Lon", ylab = "Lat", col = tim.colors())
title("Full model")
map("state", add = T)
surface(out.poly, las = 1, xlab = "Lon", ylab = "Lat", col = tim.colors())
map("state", add = T)
title("Spatial trend")
surface(out.spatial, las = 1, xlab = "Lon", ylab = "Lat", col = tim.colors())
map("state", add = T)
title("Spatial error")
```



Findings:

The `spatialProcess()` function in the `fields` package fits a Gaussian process model with the following components:

- *Mean function*: A linear model (`just.fixed = TRUE`) is used to estimate the large-scale spatial trend. This is the "spatial trend" part visualized in the middle panel of the surface plots.
- *Covariance function*: The spatially correlated residuals are modeled using an **exponential covariance function** by default in `spatialProcess()`. This captures how the correlation between values decays with distance.

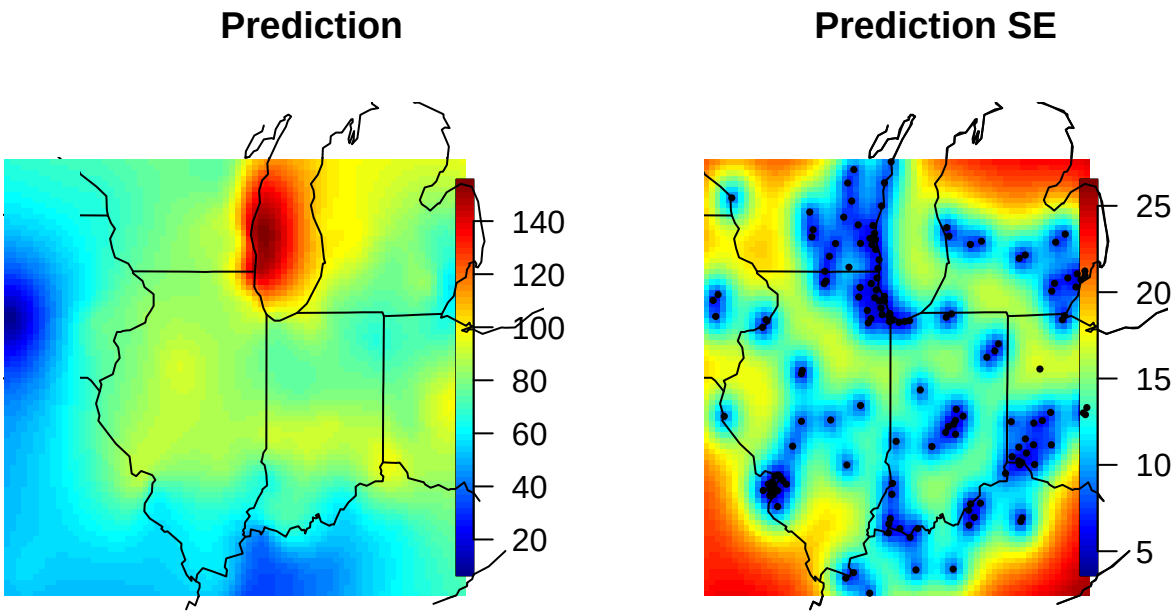
Thus, the total prediction is a sum of:

1. A fixed mean structure (linear in coordinates),
2. A zero-mean Gaussian process for the residual spatial variation.

The decomposition into "Full model", "Spatial trend", and "Spatial error" visually shows this split, where the spatial error captures more local deviations not explained by the broad trend.

3.5. Make prediction map and the associated prediction uncertainty map using the script below. Can you explain the spatial variation in the uncertainty map?

```r
xg <- fields.x.to.grid(loc[good,])
Pred <- predict.mKrig(fit, xnew = expand.grid(xg))
SE <- predictSE(fit, xnew = expand.grid(xg))

par(mfrow = c(1, 2), las = 1)
map("state", xlim = rg[, 1], ylim = rg[, 2])
image.plot(xg[[1]], xg[[2]], matrix(Pred, 80), add = T)
map("state", xlim = rg[, 1], ylim = rg[, 2], add = T)
title("Prediction")
map("state", xlim = rg[, 1], ylim = rg[, 2])
image.plot(xg[[1]], xg[[2]], matrix(SE, 80), add = T)
map("state", xlim = rg[, 1], ylim = rg[, 2], add = T)
title("Prediction SE")
points(loc, pch = 16, cex = 0.5)
```



Findings:

The uncertainty map (Prediction SE) reveals the spatial distribution of the standard errors associated with predictions from the Gaussian process model. As expected in kriging models, uncertainty is lowest in areas where observations are dense and higher in regions far from observed locations. This reflects the model's reliance on nearby data points to make predictions—greater proximity to observations reduces uncertainty. Specifically...

- Near dense clusters of observation points, the prediction standard error is small, indicating high confidence in model estimates.
- As you move away from observation locations (especially near the edges or gaps in the spatial domain), the prediction SE increases, visualized by warmer color shades (e.g., reds/oranges).
- The pattern of uncertainty is consistent with spatial interpolation theory, where model predictions become more uncertain as spatial extrapolation increases.

This map is essential for understanding not just what the model predicts, but how reliable those predictions are across the spatial domain.